HOW FRONTEND & BACKEND COMMUNICATE

SECTION 01 / ARCHITECTURE

A BEGINNER-FRIENDLY GUIDE TO PROJECT STRUCTURE, APIS, GIT & DEPLOYMENT

1. PROJECT STRUCTURE FUNDAMENTALS

When starting software development—especially with AI tools—it's easy to throw all files into a single directory. However, clean structural separation between client and server responsibility is essential for security, maintainability, and scalability.

MESSY / ANTI-PATTERN

Monolithic Confusion

```
my-project/
├── index.html
├── server.js
├── styles.css
├── db.js
├── script.js
└── package.json
```

Issues: UI and server logic are mixed. Database access tokens risk exposure to browser clients. Difficult to deploy independently.

CLEAN / BEST PRACTICE

Isolated Architecture

```
my-project/
├── frontend/
│   ├── src/
│   └── package.json
└── backend/
    ├── src/
    └── package.json
```

Benefits: Strict separation of concerns. UI cannot accidentally expose database secrets. Clean dependency boundaries.

Core Application Responsibilities

Component	Location	Primary Responsibility
FRONTEND	`/frontend`	User Interface (UI), user interactions, routing, client-side state, rendering data.
BACKEND	`/backend`	API endpoints, authentication, business logic, authorization, data validation.
DATABASE	`Isolated Server`	Stores application state, user records, persistent relational/ document data safely.

Initializing Separated Directories

To initialize this structure in a terminal, run the standard directory creation command:

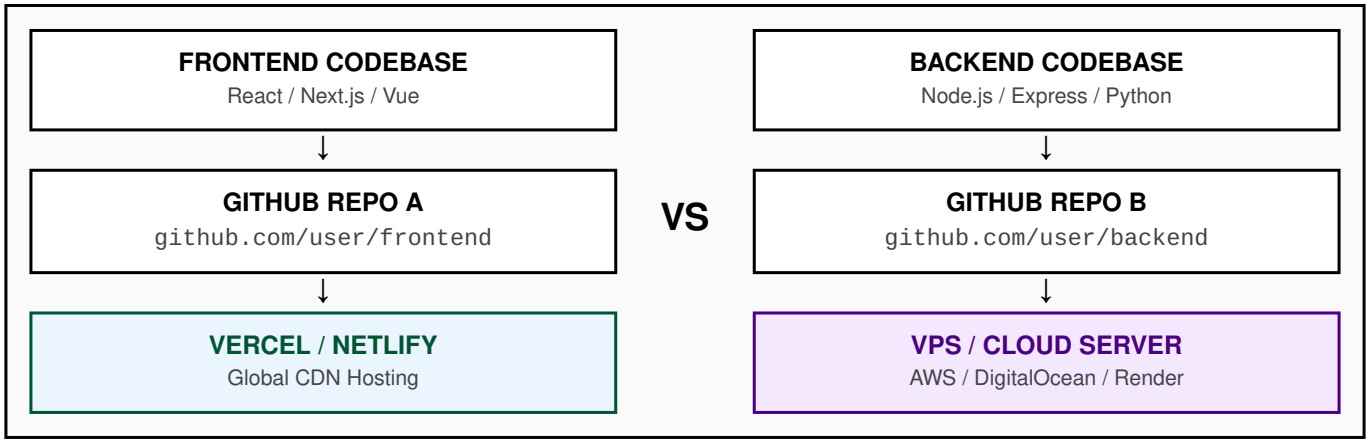
```
mkdir frontend
mkdir backend
```

Command Explanation: The `mkdir` (make directory) command creates brand new, isolated folders in your file system to house two completely separate applications.

KEY CONCEPT **Why Structure Matters:** Good project structure enforces clear boundary lines between what runs publicly on a user's browser (Frontend) and what runs securely on your server (Backend). It makes your codebase easier to understand, maintain, test, and scale over time.

2. REPOSITORIES & CLOUD DEPLOYMENT

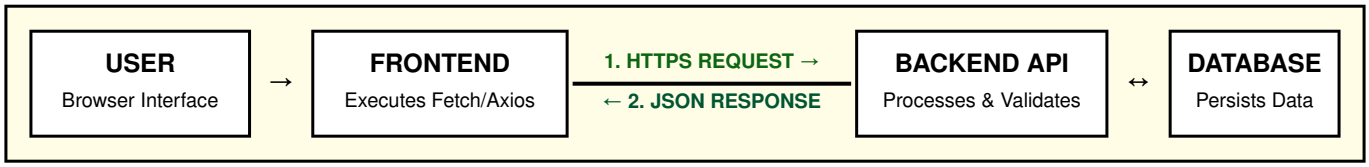
Frontend and backend applications do not need to live in the same repository or sit on the same server physical hardware.



- **Separate Repositories are Optional:** You can keep them in one repository (Monorepo) or separate repositories (Polyrepo). Separate repos allow independent continuous integration, deployment pipelines, and version control.
- **Network Independence:** Even across different repositories and physical cloud hosting platforms, frontend and backend communicate seamlessly across standard internet protocols (HTTP/HTTPS).

3. THE REQUEST-RESPONSE LIFECYCLE

The frontend **never accesses backend source files or databases directly**. All interaction happens by sending structured HTTP requests to exposed backend API endpoints over the network.



Step-by-Step Execution Flow:

1. **User Action:** A user clicks a button or loads a screen on the frontend application.
2. **HTTPS Request:** Frontend fires an asynchronous HTTP request (e.g., `GET`, `POST`) to an API URL endpoint.
3. **Backend Processing:** The backend server intercepts the request, verifies authentication/tokens, and executes business logic.
4. **Database Query:** If data is required, backend queries the database, receives raw records, and formats them.
5. **JSON Response:** Backend returns a structured status code (e.g., `200 OK`) and data payload, typically formatted as **JSON**.
6. **UI Update:** Frontend parses the incoming JSON data and dynamically re-renders the UI for the user.

Real-World API Example

Frontend triggers a request to fetch user records:

```
GET https://api.example.com/api/users
```

Backend processes request and replies with JSON payload:

```
{
  "status": "success",
  "users": [
    { "id": 101, "name": "Rafi", "role": "Developer" }
```

```
  ]
}
```

IMPORTANT RULE **Security Boundary:** The frontend cannot read backend files, database tables, or backend server code. It can only talk through public, explicit API endpoints created by the developer.

4. SERVER FAILURE SCENARIOS

Because Frontend (e.g., Vercel) and Backend (e.g., Cloud VPS) run on separate infrastructure, individual server failure is isolated.

STATUS: RUNNING

Frontend (Vercel CDN)

- ✔ Fully operational.
- ✔ Static assets (HTML/CSS/JS) render instantly.
- ✔ Base UI pages load without issue.

STATUS: CRASHED

Backend (Cloud VPS)

- ✖ Server offline or crashed.
- ✖ API requests time out or return 500 error.
- ✖ Dynamic data features fail to load.

Takeaway: If backend crashes, the frontend UI does not vanish or crash automatically. However, features requiring backend data (login, payments, data feeds) will fail or display fallback error states.

5. CORS & ENVIRONMENT VARIABLES

Cross-Origin Resource Sharing (CORS)

Browsers restrict HTTP requests initiated from scripts to a different origin (domain, protocol, or port) than the one serving the frontend document.

- **Browser Security:** CORS is enforced by the *browser* to protect users, preventing malicious sites from reading sensitive data from another domain.
- **Configuration:** If Frontend is at `app.example.com` and Backend is at `api.example.com`, the Backend must explicitly send headers allowing the frontend origin:

```
Access-Control-Allow-Origin: https://app.example.com
```

NOTE **CORS is Security, NOT Authentication:** CORS prevents browsers from making unapproved cross-origin requests. It does not replace user login, JWT tokens, or API keys.

Environment Variables

Instead of hardcoding URLs, frontend applications store the target backend API host in environment configuration files:

```
NEXT_PUBLIC_API_BASE_URL=https://api.example.com
```

Code usage example:

```
const response = await fetch(`${process.env.NEXT_PUBLIC_API_BASE_URL}/api/users`);
```

SECURITY WARNING **Never Leak Secrets:** Anything prefixed for frontend bundling (like `NEXT_PUBLIC_` or `REACT_APP_`) is embedded directly into the public JavaScript bundle. **NEVER put database passwords, secret keys, or private tokens in client environment files!**

6. GIT BRANCHING MODEL

In team software engineering, Git branches isolate changes inside a repository before merging them to main production code.

```
main (Production Code)
├── feature/login       (Isolated Login Work)
├── feature/payment     (Isolated Stripe Integration)
└── feature/dashboard   (Isolated Dashboard UI)
```

CONCEPT COMPARISON

REPOSITORY: Entire codebase container (e.g., `frontend` folder). Stores complete history and project assets.

BRANCH: An isolated development line inside a repository. Allows testing features without breaking main code.

ARCHITECTURE SUMMARY & INTERVIEW GUIDE

7. SYSTEM ARCHITECTURE CHEAT SHEET

FRONTEND
UI + Client-Side Logic

→

API LAYER
HTTP / HTTPS Protocols

→

BACKEND
Business Logic + Database

GitHub Repository	Stores and version-controls the application source code.
Git Branch	Isolates specific feature work from main production branch.
Deployment Host	Runs the built application on cloud servers (e.g., Vercel, AWS).
API Endpoint	Secure gateway url where requests are received and answered.
HTTP / HTTPS	Protocol carrying requests (data lookup) and responses (JSON payloads).

8. TECHNICAL INTERVIEW QUESTIONS & ANSWERS

Q1: Where is your frontend application located?
Answer: In its own frontend codebase/repository, compiled and served via a CDN (e.g., Vercel) to client browsers.

Q2: Where is your backend application located?
Answer: In a separate backend codebase/repository running on a cloud server or container host (e.g., AWS, VPS).

Q3: How do frontend and backend communicate across separate repositories?
Answer: Via asynchronous HTTP/HTTPS network API requests (using tools like `fetch` or `axios`), exchanging JSON data.

Q4: What happens to the frontend if the backend goes down?
Answer: Static frontend pages still render, but features dependent on backend APIs will fail or display fallback error states.

Q5: Why use Git branches during development?
Answer: To isolate feature development, bug fixes, and testing so unverified changes don't break the stable main production code.

Q6: What is CORS and why is it necessary?
Answer: CORS is a browser security mechanism that restricts cross-origin HTTP requests, ensuring servers explicitly permit frontend origins.

"AI can help you build faster.
Understanding what you built helps you build better."

ORGANIZE. ISOLATE. UNDERSTAND. BUILD.